Comparison of Lightweight Encryption Algorithms

B.Tech Final Year Project

Rajesh Kumar Jena (B122088) Soumil Kumar (B122114) Punit Kumar (B422041)

International Institute of Information Technology
Bhubaneswar
Supervisor: Prof. Debasish Jena

May 2026

Outline

Motivation and Problem Statement

Historical Context

- DES (1977) $\rightarrow$ AES (2001): Evolved for desktop/server computing
- IoT devices: constrained CPU, memory, power makes traditional algorithms impractical
- Mirai botnet (2016): Exploited insecure embedded devices, with millions hijacked

Problem

- Developers deploy weakened or ad hoc security
- Lightweight cryptography: efficient primitives for constrained platforms
- Need to balance security, performance, and implementation cost

Objectives & Contributions

This study aims to:

- Evaluate PRESENT, SPECK, and ASCON on Raspberry Pi Pico (ARM Cortex-M0+)
- Compare execution time, memory usage, and throughput
- Identify design trade-offs for constrained embedded environments

Contributions:

- Reproducible Rust-based benchmarking framework for RP2040
- Empirical data across block cipher, mode, and payload-size dimensions
- Practical deployment recommendations based on measured trade-offs

Algorithms: Design Properties

Property	PRESENT	SPECK	ASCON
Block Size	64 bits	64 bits	64 bits
Key Sizes	80, 128 bits	64–256 bits	128 bits
Structure	SPN	ARX GFN	Sponge-Duplex
Rounds	31/32	22–34 (key size)	12
Standardization	ISO 29192-2	—	NIST SP 800-232
AEAD Support	No	No	Yes

Key Differentiators:

- **PRESENT**: ∼1,570 GE (**hardware-optimized**)
- **SPECK**: **42 CPB** (**software fastest**)
- **ASCON**: **AEAD** + hashing built-in (NIST standard)

Platform & Metrics

Hardware: Raspberry Pi Pico (ARM Cortex-M0+, 133 MHz, 264 KB SRAM, 2 MB Flash)

Metrics:

- **Performance**: Throughput (Mbps), Cycles/Byte (CPB), latency
- **Memory**: ROM footprint (KB), RAM usage (bytes)
- **Energy**: µJ per byte using V-I-t model (I = 80–90 mA per RP2040 datasheet; estimates are relative for comparison)

Testing Procedure:

- 1000+ iterations per cipher-mode-payload combination
- Hardware timer (RP2040 internal counter @ 133 MHz)
- Test vector validation (NIST vectors for PRESENT, Beaulieu for SPECK)
- Payload sizes: 8B, 16B, 64B, 256B, 1024B

Block Cipher Benchmarks

Algorithm	CPB	Mbps
PRESENT-80	2,958	0.34
PRESENT-128	3,068	0.33
SPECK-64/96	42	23.81
SPECK-64/128	42	23.67
ASCON-128	132	7.56

Key Finding: SPECK is **70x faster** than PRESENT and **3x faster** than ASCON (23.67 Mbps vs 0.34 Mbps vs 7.56 Mbps).

Insight: Single-block throughput shows the dramatic performance gap: SPECK's ARX design dominates on software platforms.

Mode Impact: ECB vs CBC vs CTR

Cipher Mode	64B (Mbps)	256B (Mbps)	Trade-off
SPECK-ECB	**19.04**	**20.33**	**Fastest**; no IV overhead
SPECK-CBC	16.07	18.90	−16%; sequential dependency
SPECK-CTR	17.14	19.42	−11%; parallelizable, semantic security
PRESENT-ECB	0.34	0.34	Fastest; block-aligned
PRESENT-CTR	0.33	0.33	∼0% overhead (negligible vs 2,958 CPB)

Key Insight: Mode overhead is **cipher-dependent**. SPECK loses 11–16% in CBC/CTR; PRESENT's overhead is **negligible** due to dominant S-box cost.

Recommendation: Use **CTR** for applications needing semantic security and parallelism without sacrificing performance.

Why This Performance Gap?

Architectural Explanation:

- **SPECK (ARX)**: Add, Rotate, XOR → **single-cycle ARM** instructions (`ADDS`, `RORS`, `EORS`), no memory lookups
- **PRESENT (SPN)**: Bit-permutation requires **64 separate** bit extractions/reinsertions per round, 31/32 rounds = **70x overhead**
- **ASCON (Sponge)**: 64-bit words align with Cortex-M0+ datapath, but 12-round p^a permutation **heavier** than SPECK's 5-instruction round

Bottom Line: **Hardware-optimized** ciphers (PRESENT) **sacrifice software**; **software-optimized** ciphers (SPECK) **exploit** architectural fit.

Memory Footprint

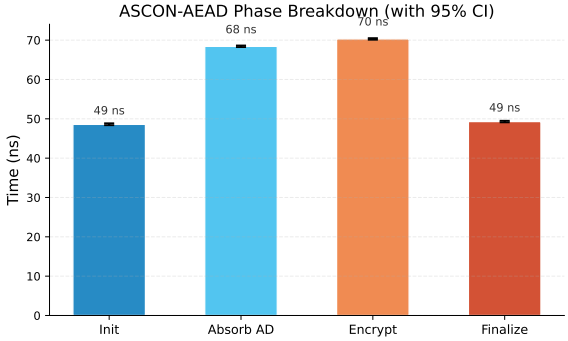
Cipher	Key	State	ROM	RAM
PRESENT-80	80 bits	8 bytes	∼2 KB	∼16 B
PRESENT-128	128 bits	8 bytes	∼2.5 KB	∼16 B
SPECK-64/96	96 bits	16 bytes	∼1 KB	∼32 B
SPECK-64/128	128 bits	16 bytes	∼1.2 KB	∼32 B
ASCON-128	128 bits	40 bytes	∼3 KB	∼40 B

SPECK: Smallest ROM (∼1 KB) from ARX-only ops, no lookup tables

ASCON Phase Breakdown

Phase	Cycles
Initialization	427
Absorb (AD)	216
Encrypt	464
Finalize	422
Total	**1,529**

Why init $\approx$ finalize? Both run full 12-round p^a permutation; absorb/encrypt share similar per-byte cost.



Energy Efficiency (µJ/byte)

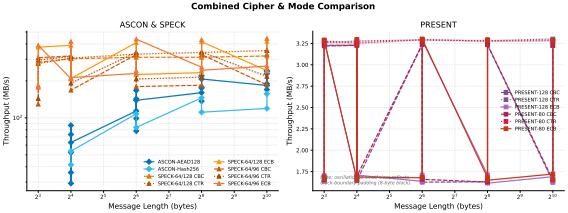
Cipher	16B	64B	256B	1024B
SPECK-64/128 (CTR)	0.92	0.85	0.82	0.79
ASCON-128 (AEAD)	8.08	2.02	1.60	1.48
PRESENT-128 (CTR)	85.56	85.29	85.51	84.96

- **SPECK**: **108x** more energy-efficient than PRESENT (ratio invariant to current assumption)
- **ASCON**: ∼129 µJ initialization overhead (constant at 16–64B), then scales from 8.08→1.48 µJ/byte
- Energy estimated at 80–90 mA active current; **relative ratios valid** for deployment decisions
- *Ideal for battery-powered IoT sensors*

Speedup Analysis

Performance relative to ASCON:

Cipher	64B	256B	1024B
SPECK (ECB)	1.34x	1.08x	1.03x
PRESENT	0.012x	0.010x	0.009x



Insight: ASCON's 427-cycle init overhead amortizes at larger payloads; SPECK advantage shrinks from 34% to 3%.

Note on PRESENT: Oscillation at small sizes reflects 8-byte block padding overhead versus cipher cost.

Scorecard Comparison

Metric	PRESENT	SPECK	ASCON
Throughput (Mbps)	0.34	**23.67**	7.56
CPB (cycles)	2,958	**42**	132
ROM (KB)	2.5	**1.2**	3.0
RAM (bytes)	16	32	40
AEAD	No	No	✓
NIST Standard	No	No	✓
ISO Standard	✓	No	No

Summary: SPECK dominates **performance** and **ROM**; ASCON provides **standardization and AEAD**; PRESENT excels in **hardware** but **poor software**.

Security Considerations

- **ASCON**: **NIST SP 800-232**, proven indifferentiability bounds, no attacks on full 12-round permutation **strongest**
- **PRESENT**: ISO/IEC 29192-2, **provable SPN** bounds (max diff probability 2^{-2}), confidentiality only (no authentication)
- **SPECK**: **No provable** bounds, best attacks reduce 20/27 rounds; ISO rejection (NSA origin, geopolitical objections) (*research use only*)
- **Side-channels**: **SPECK** vulnerable to CPA (50 traces on 8-bit); **PRESENT's** constant-time ops more resistant; **ASCON** shows moderate resistance

Trade-off: ASCON offers strongest security; PRESENT offers provable bounds; SPECK is fastest but has weakest guarantees.

Key Takeaways: Design Lessons

Algorithm Design Trade-offs:

- **Hardware $\neq$ Software**: PRESENT's bit-permutation is elegant in silicon (1,570 GE) but catastrophic in code (2,958 CPB)
- **Instruction Set Alignment**: SPECK's ARX operations map **perfectly to ARM** instructions (42 CPB); sponge-based designs add fixed overhead (427 cycles init)
- **AEAD vs Raw Cipher**: ASCON's built-in authentication justifies overhead; SPECK+HMAC would be bulkier

For Practitioners:

- Choose by **workload**, not by raw speed: **sensor data** (SPECK) versus **authenticated IoT** (ASCON) versus **legacy** (PRESENT)
- **Standardization matters**: NIST-endorsed ASCON gaining adoption; SPECK rejected by ISO (politics and no provable bounds)
- **Context is everything**: RP2040 (Cortex-M0+) results don't generalize to hardware, 8-bit MCUs, or server environments

Use-Case Recommendations

1. **Real-time sensor data**: SPECK-64/128 (CTR), 23.67 Mbps, 32B RAM
2. **Secure IoT with auth**: ASCON, 7.56 Mbps, AEAD, NIST standardized
3. **RFID tags**: PRESENT, ultra-low gate count (1570 GE)
4. **Firmware authentication**: ASCON-hash, 320-bit state, 12-round p^a
5. **Hybrid deployment**: SPECK (bulk) + ASCON (auth tags)
6. **Legacy integration**: PRESENT-128 (CBC) for backward compatibility

Limitations & Future Work

Current Limitations:

- **Single platform**: RP2040 only, architecture-specific results
- **Single language**: Rust implementation, no C/C++ comparison
- **No side-channel analysis**: Timing and power attacks not evaluated

Natural Extensions of This Work:

1. Port implementations to ESP32 and compare against RP2040 results
2. Implement SPECK in C with compiler optimizations and benchmark against Rust
3. Conduct comparative energy analysis using actual multimeter measurements
4. Expand to test multiple RP2040 clock speeds (50 MHz, 100 MHz, 133 MHz)
5. Compare ASCON-128 throughput against ASCON-128a on same platform

Thank You